Compiler Construction Lab - Assignment 7

Name: Dev Vachhani
PRN.: 24070122517
Batch: B3

Objective

Write a program using ply.lex and ply.yacc to evaluate a given postfix expression.

Use integer/float operands and binary operators +, - , * and /.

Theory

Postfix notation, also called Reverse Polish Notation (RPN), places each operator after its operands. For example, the infix expression (3 + 4) * 2 is written in postfix form as 3 4 + 2 *. Parentheses and operator precedence are not needed because the order of the operands and operators determines how the expression is evaluated.

This program uses **PLY** (Python Lex-Yacc), a Python implementation of the Lex and Yacc tools:

- **ply.lex** is the lexical analyzer. It recognizes integer and floating-point numbers, along with the binary operators +, -, *, and /.
- **ply.yacc** is the parser. Its grammar describes valid postfix expressions and uses semantic actions to calculate results as the expression is parsed.

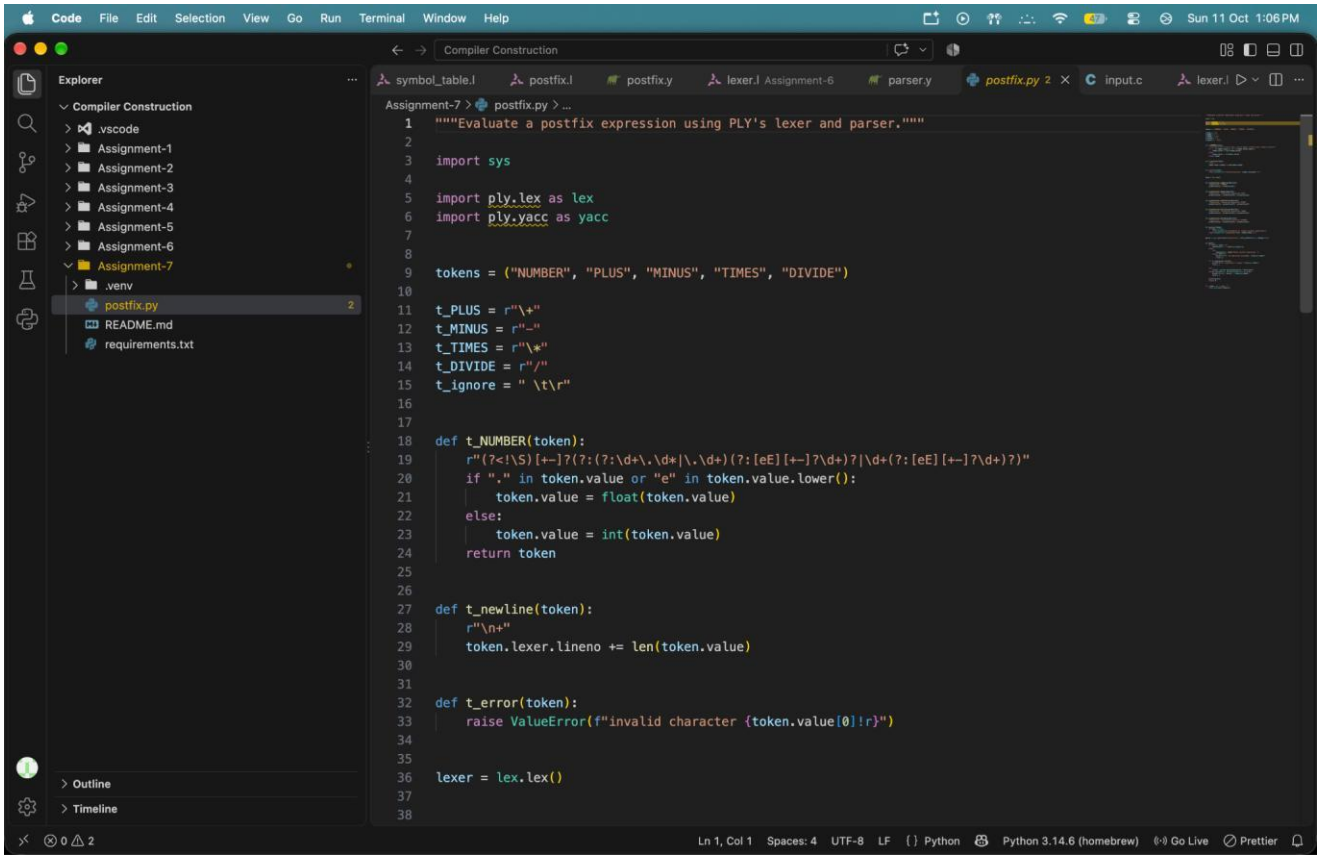
The grammar has a rule for a number and a rule for each binary operation. In general, a postfix expression has the form:

- expression → NUMBER
- expression → expression expression operator

When the parser recognizes two expressions followed by an operator, it applies that operator to their values. For example, in 3 4 +, it parses 3 and 4 as expressions, then adds them when it reaches +. The result, 7, becomes an expression that can be used in the next operation. Thus, 3 4 + 2 * evaluates to 14.

Integer operands are stored as integers, while decimal or exponent-form operands are stored as floating-point numbers. The division uses Python's / operator, which produces a floating-point result. The program reports errors for invalid characters, malformed expressions, and division by zero.

Screenshots and Execution Results



```python
"""Evaluate a postfix expression using PLY's lexer and parser."""

import sys

import ply.lex as lex
import ply.yacc as yacc


tokens = ("NUMBER", "PLUS", "MINUS", "TIMES", "DIVIDE")

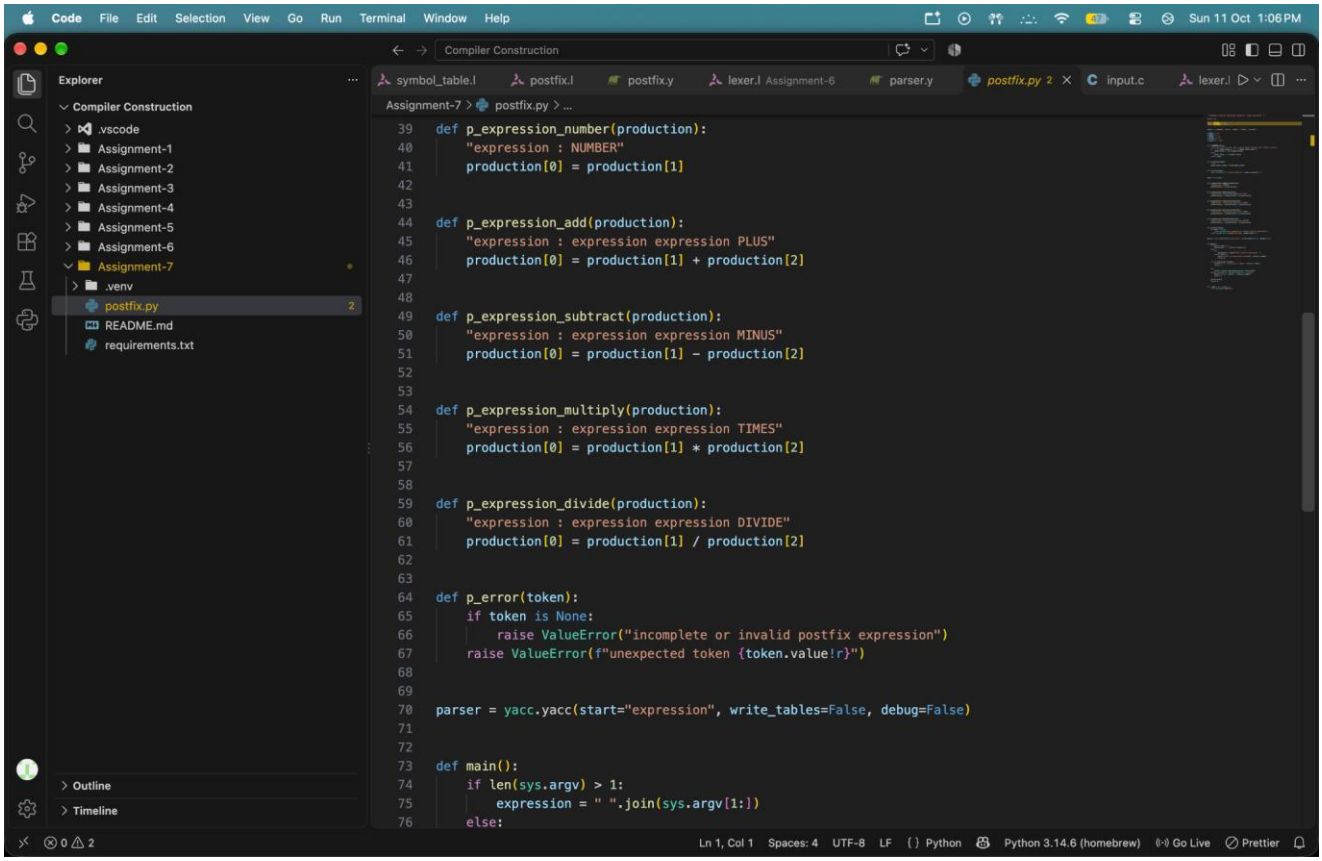
t_PLUS = r"\+"
t_MINUS = r"-"
t_TIMES = r"\*"
t_DIVIDE = r"/"
t_ignore = " \t\r"


def t_NUMBER(token):
    r"(?<!\S)[+-]?(?:(?:\d+\.\d*|\.\d+)(?:[eE][+-]?\d+)?|\d+(?:[eE][+-]?\d+)?)"
    if "." in token.value or "e" in token.value.lower():
        token.value = float(token.value)
    else:
        token.value = int(token.value)
    return token


def t_newline(token):
    r"\n+"
    token.lexer.lineno += len(token.value)


def t_error(token):
    raise ValueError(f"invalid character {token.value[0]!r}")


lexer = lex.lex()
```



```python
def p_expression_number(production):
    "expression : NUMBER"
    production[0] = production[1]


def p_expression_add(production):
    "expression : expression expression PLUS"
    production[0] = production[1] + production[2]


def p_expression_subtract(production):
    "expression : expression expression MINUS"
    production[0] = production[1] - production[2]

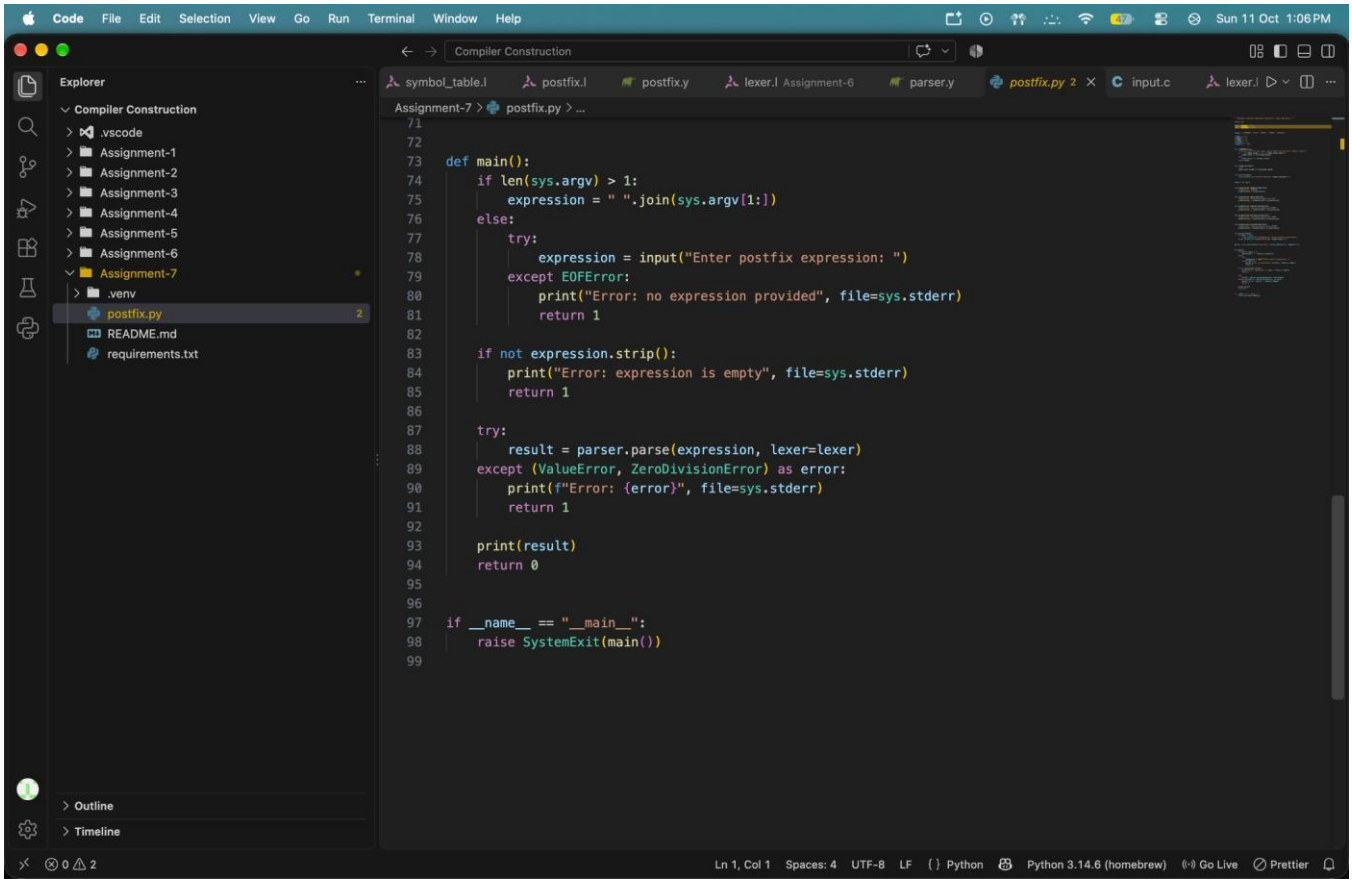

def p_expression_multiply(production):
    "expression : expression expression TIMES"
    production[0] = production[1] * production[2]

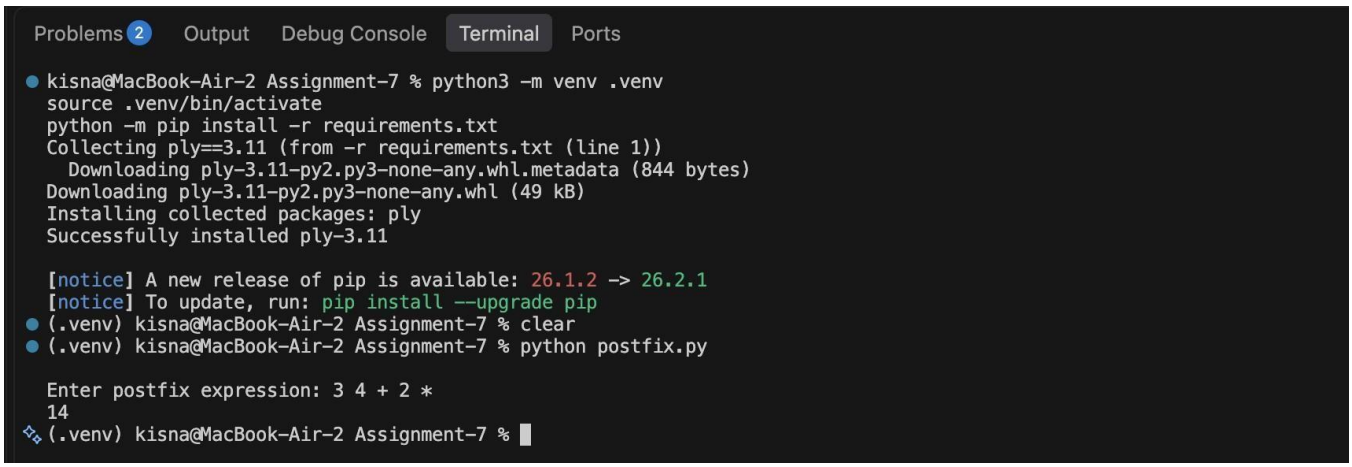

def p_expression_divide(production):
    "expression : expression expression DIVIDE"
    production[0] = production[1] / production[2]


def p_error(token):
    if token is None:
        raise ValueError("incomplete or invalid postfix expression")
    raise ValueError(f"unexpected token {token.value!r}")


parser = yacc.yacc(start="expression", write_tables=False, debug=False)


def main():
    if len(sys.argv) > 1:
        expression = " ".join(sys.argv[1:])
    else:
```


Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:06 PM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
.venv
postfix.py
README.md
requirements.txt
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y
postfix.py 2
input.c
lexer.l
Assignment-7 > postfix.py > ...
def main():
if len(sys.argv) > 1:
expression = " ".join(sys.argv[1:])
else:
try:
expression = input("Enter postfix expression: ")
except EOFError:
print("Error: no expression provided", file=sys.stderr)
return 1
if not expression.strip():
print("Error: expression is empty", file=sys.stderr)
return 1
try:
result = parser.parse(expression, lexer=lexer)
except (ValueError, ZeroDivisionError) as error:
print(f"Error: {error}", file=sys.stderr)
return 1
print(result)
return 0
if __name__ == "__main__":
raise SystemExit(main())
Outline
Timeline
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Python 3.14.6 (homebrew) Go Live Prettier


Problems 2
Output
Debug Console
Terminal
Ports
kisna@MacBook-Air-2 Assignment-7 % python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
Collecting ply==3.11 (from -r requirements.txt (line 1))
Downloading ply-3.11-py2.py3-none-any.whl.metadata (844 bytes)
Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
Installing collected packages: ply
Successfully installed ply-3.11
[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(.venv) kisna@MacBook-Air-2 Assignment-7 % clear
(.venv) kisna@MacBook-Air-2 Assignment-7 % python postfix.py
Enter postfix expression: 3 4 + 2 *
14
(.venv) kisna@MacBook-Air-2 Assignment-7 %